

CRLF-Powered Desync Attacks: Beheading HTTP Streams

CRLF-Powered Desync Attacks: Beheading HTTP Streams - Tom Stacey, t0xodile's blog.

- Published: 2026-05-20
- Original: <<https://thomas.stacey.se/posts/CRLF-Powered-Desync-Attacks/>>
- Preserved from: <https://thomas.stacey.se/posts/CRLF-Powered-Desync-Attacks/> (stored) on 2026-08-14
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Coming to BlackHat USA and DEFCON 2026...

[Abstract](#)

Have you ever discovered a header injection vulnerability and settled for little more than an open redirect or XSS? In this Briefing, we will introduce a battle-tested "header injection" powered desync methodology, enabling you to perform HTTP request smuggling attacks against even strictly RFC-compliant proxy chains.

We will begin by explaining a well-known but overlooked CRLF injection primitive that produced HTTP Request Splitting inside the core infrastructure of a major CDN, resulting in the capture of live users' credentials across thousands of compromised applications.

Building upon this, we'll demonstrate how header injections can be used to exploit more traditional smuggling attack classes, even when no parser discrepancy exists. Finally, we'll reveal how you can shift previously non-compliant desync attacks into the browser, unlocking a plethora of novel exploitation opportunities even when keep-alive connections are not shared between users. The result is a slew of real-world case studies with impacts ranging from account takeovers via desync-enabled XSS gadgets to cache poisoning, response queue poisoning, access control bypasses, and in several cases the possibility of creating the ever-terrifying desync worm.

To complement our methodology and case studies, we'll share our research journey and release two open-source tools that introduce robust detection of header injections regardless of your proxy of choice.

** [Research](#)

** [bug-bounty](#)

This post is licensed under [CC BY 4.0](https://creativecommons.org/licenses/by/4.0/) by the author.

Archived from <https://thomas.stacey.se/posts/CRLF-Powered-Desync-Attacks/>. Rendered offline from the local Markdown copy.